

FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

July 25, 2026

Abstract

Semantic caches are evaluated on hit rate, latency, and at best false-hit rate, none of which distinguishes a correct hit from a stale one. We name and measure that gap directly: *stale-hit-rate*, the fraction of served cache hits whose content was no longer valid at serve time. We extend GPTCache, a frozen and widely used open-source semantic cache, with a per-cluster staleness model that is fit from observed data and consumed in two places: a validity gate on the serve path, and a soft decay term in eviction. Across a bracketing experiment, a six-row ablation, and three parameter sweeps, all run with ten seeds per configuration on a deterministic synthetic workload, we find that the validity gate cuts stale-hit-rate by roughly fifteen times relative to a frequency-based floor, that the learned decay rate reliably outperforms a single pooled rate and tracks an oracle rate closely, and that this correctness gain comes at a bounded and quantified cost in hit rate. We report the size of that trade-off directly rather than presenting the extension as a uniform win.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call, and serves a stored response when an incoming query is close enough in embedding space to one it has already answered. The promise is simple: skip the call, return the same answer faster and for free. The problem is that “the same answer” is a claim about time as well as similarity, and most semantic caches never check the time part.

This gap sits at the center of a growing class of production systems that place a large language model between a user and an answer about frequently changing content. Consider a system whose job is to monitor how a brand is represented when people ask AI assistants about it, across many brands, geographies, and models — the kind of workload behind answer engine optimization (AEO), a discipline that exists specifically because brand visibility inside AI-generated answers, rather than a ranked list of links, is now a measurable and contested surface. The set of distinct questions such a system fields is highly repetitive, the same style of question asked over and over as users onboard or monitoring jobs re-run on a schedule, which is exactly the shape of workload a semantic cache absorbs, and absorbing it cheaply matters since every answered query carries a real dollar cost that scales with the number of brands and questions tracked.

The underlying facts these questions are about do not sit still: a product goes out of stock, a price changes, a review’s sentiment shifts, and the correct answer to an unchanged question changes with it. A cache that only asks “have I seen a similar enough question before” and never asks “is that answer still true” will keep serving a stale answer indefinitely once written, and the system on top of it has no way to distinguish a correct hit from an incorrect one, because the metric that would, stale-hit-rate, does not exist in any standard cache evaluation. Answer-facing systems built on language models need a caching layer that treats correctness over time as a first-class property, not an afterthought bolted onto count-based eviction.

Two concrete gaps follow, and motivate the two components this project adds.

No staleness concept in the serve path. A cached answer about a fact that changes on its own schedule is served indefinitely once written; “found in cache” is silently treated as “correct,” with no mechanism to ever reconsider that judgment.

No cost term in eviction. Count-based policies such as LRU and LFU treat two entries with identical access patterns identically, even when one is far more expensive to regenerate. A policy blind to regeneration cost will evict a rarely accessed but expensive entry ahead of a cheap one accessed once more recently, the wrong trade from a cost-minimization standpoint.

Industry and research reflect this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock time-to-live and no similarity matching, sidestepping the freshness question by making every key an exact match. Semantic-caching products use a single fixed global similarity threshold with plain time-to-live or count-based eviction, with no notion of per-topic staleness. The research literature, discussed next, is further ahead, having proposed cost-aware and even staleness-aware eviction formulas, but none of it names or measures stale-hit-rate as an evaluation metric in its own right. This project positions itself against that literature, not the weaker product baseline.

1.2 Related work

The closest prior art, and this project’s primary eviction comparator, is Biton and Friedman’s work on semantic-cache eviction[1], which proves optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality. It evaluates these on GPTCache and releases its implementation, part of why GPTCache was chosen as this project’s baseline. Two of its findings shape this project’s design directly: LRU is a weak policy on semantic workloads and frequency-based policies are a much stronger baseline, which is why every comparison here treats an LFU-class policy, not LRU, as the floor. The delta against FreCoS: their policies combine access-pattern signals only, while this project adds two signals orthogonal to access pattern, regeneration cost and content validity over time, and evaluates against a correctness metric, stale-hit-rate, their setting does not define.

The structurally closest published eviction formula is Cortex’s LCFU policy[2], whose earlier preprint circulated under the name Asteria and is cited here by its current title. LCFU scores an entry as a product of log-scaled frequency, cost, latency, and a staleness term, divided by size, with a time-to-live purge. The delta is narrow: Cortex’s staleness term is a static “staticity” score assigned by a language model at write time, while FreCoS’s is a decay rate learned per cluster from observed staleness. The distinction is learned against assigned, and the bracketing experiment in Section 5.1 tests whether that learning does measurable work. Cortex’s formula was not reimplemented, since it incorporates remote-tool-call latency metadata with no equivalent here.

Several other lines of work bear on individual pieces of the design without matching its combination. GDSF[3] scores entries as cost times frequency divided by size with a global-clock aging term, but that term ages access recency, not content validity, so it has no freshness semantics. Category-Aware caching[4] assigns per-category time-to-live values that are load-based rather than learned, and defines no stale-hit-rate equivalent; it is also why a third, originally planned component of this project, calibrated per-cluster similarity thresholds, was cut, since it would have been a lossy special case of both this work and per-embedding threshold learning. FreshCache[5] shares this project’s lineage of putting staleness in the serve path, but as a probabilistic risk budget per tier rather than a hard gate, and is listed as future work as an alternative framing. SCALM[6] clusters cache entries similarly to this project’s k-means step, but its significance score is not staleness-aware. MeanCache[7] is cost-motivated but does not target eviction. W-TinyLFU[8] is an admission-control policy, deciding whether an entry is

worth admitting before eviction is invoked, and is out of scope since it would add an orthogonal axis rather than test either of this project’s contributions.

The alternative baseline vCache[9] was considered and rejected: its contribution is per-prompt threshold learning, replacing a single global threshold with one learned per embedding, and eviction is explicitly not its focus. Building this project’s staleness and cost work on top of it would layer a new contribution on a system whose own novelty sits elsewhere, diluting rather than sharpening the comparison.

2 Baseline

2.1 Why GPTCache

This project extends GPTCache[10] rather than a more actively maintained semantic cache, and the case rests on architecture and control properties rather than community activity. GPT-Cache’s most recent release is v0.1.44 (August 2024); it carries dozens of open issues and pull requests, and the maintainers state that support for new integrations has stopped. Ordinarily this counts against a dependency; here it is an advantage, since a frozen interface is a better scientific control than a moving one — the baseline measured early in this project is byte-identical to the one measured at the end, and every delta reported below is attributable to the extension, not an upstream change mid-project.

The second reason is architectural fit, verified by reading the vendored source directly. Every component this project modifies, the similarity threshold check, the hit and miss decision, and the eviction registry, sits behind a registered, named interface, so the extension is additive configuration: a new eviction subclass registered through the existing hook, a metadata field added via composition around storage, and a serving-path gate plugged into one shared decision helper. Nothing vendored is edited. Every claim about GPTCache’s internal behavior quoted here was re-verified against the exact pinned commit vendored in this project’s repository, with file and line recorded.

Third, GPTCache is the baseline the closest prior art already uses, making Biton and Friedman’s released policy usable directly as this project’s primary comparator rather than requiring reimplementation. Fourth, its default embedding backend runs on the CPU deterministically, keeping every benchmark run reproducible without a GPU dependency or nondeterminism from a hosted embedding service.

2.2 GPTCache’s behavior and default eviction policy

GPTCache serves a cached response when an incoming query is semantically close enough to a previously seen one, using a single global cosine similarity threshold, 0.8 by default, rather than exact-match keys. Its eviction policies are purely count-based: least-recently-used, least-frequently-used, first-in-first-out, or random replacement. None reads generation time, access recency, or regeneration cost beyond the single counter each already tracks, and none retains metadata on an evicted entry; the eviction cache stores only a boolean flag per key. Creation and last-access timestamps are persisted per entry but never read by any decision logic, existing as columns with no downstream consumer, precisely the gap this project’s extension repurposes. There is no admission control, every miss is written back unconditionally, and no cost accounting, false-hit tracking, or staleness tracking exists anywhere in the built-in reporting layer, which counts only per-stage operation timers.

3 Design

3.1 Contribution

The contribution is one learned artifact with two consumers. FreCoS fits a per-cluster staleness model and uses it at both ends of the cache’s lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The metric that makes both uses visible is stale-hit-rate: the fraction of served hits whose content was already invalid when served.

Cluster assignment is a fixed, seeded k-means clustering over cached-query embeddings, run once at cache-build time, with no online re-clustering, a deliberate reproducibility choice. Cluster identity, once assigned, is consumed by both the gate and the eviction policy and never recomputed independently by either, which matters because a gate and an eviction policy that disagree about an entry’s cluster is one of the most likely integration failures in a system with this shape.

For each cluster, a decay rate is fit from observed staleness in a held-out calibration split, and a time-to-live is derived at a configured confidence level. Three interchangeable modes produce this table with the same output shape: *learned* fits each cluster’s rate independently, falling back to a pooled rate below thirty observations; *global* pools every cluster into one shared rate; *oracle* takes the rate from ground truth, an upper bound never available in a real deployment.

The validity gate is the first consumer. On a candidate hit, if the entry’s age exceeds its cluster’s time-to-live, it is treated as a miss, the answer is regenerated, and the event counts as a stale hit prevented rather than served. This is a single comparison against creation time; the gate never reads last-access time, since staleness is a property of when an answer was generated, not how recently it was requested, and a popular but outdated entry must not be treated as fresh simply because it is popular.

The eviction value function is the second consumer:

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen_cost}) \cdot e^{-\lambda_c \cdot \text{age}} / \text{size_bytes} \quad (1)$$

where age is measured from creation time, never last access; κ log-compresses regeneration cost so a heavy-tailed cost distribution cannot dominate the other terms; and the victim is the entry with the lowest value, ties broken by oldest creation time then lowest entry identifier.

One detail reflects a real design tension rather than a footnote. The original frequency term was $\log(1 + \text{freq})$, exactly zero for a never-accessed entry. Since the value function is a product of four terms, a zero frequency term zeroes the entire score regardless of cost, freshness, or size, meaning a freshly written entry would always be evicted next. A cold-start regression test caught this, and the fix, $\log(2 + \text{freq})$ in Equation 1, preserves the same concave shape while guaranteeing a strictly positive score at zero frequency. It was applied throughout every experiment in this report; disclosing it here is more convincing than hiding it.

Because the gate already refreshes anything past its time-to-live, the decay term inside the value function is not itself performing staleness prevention; it acts as a soft prior favoring entries with more remaining useful life among candidates the gate already judged fresh. This is the intended division of labor, and the ablation in Section 5.2 tests it directly.

Eviction remains entry-count budgeted, matching GPTCache’s convention. Entry size in bytes is recorded and used only inside the value function’s denominator; no separate byte-budget eviction loop was built.

3.2 Cut scope

An earlier design included a third component, calibrated per-cluster similarity thresholds, cut before implementation began. It turns out to be a lossy special case of two things already in

the literature, vCache’s per-embedding threshold learning and Category-Aware caching’s per-category treatment, so building a third, weaker version exceeded what it would have contributed. It is listed as future work.

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator built for this project. A second workload, derived from Wikipedia revision history with staleness ground truth from edit timestamps, was investigated in a timeboxed feasibility spike before other implementation work began. That spike sampled thirty question-answer pairs from SQuAD 2.0[11], pulled each answer’s revision history from the Wikipedia API, and applied an automatic invalidation rule comparing each revision’s answer sentence to the one before it. Twenty cases were hand-labeled and compared against the rule’s output; agreement was 40 percent, well short of the 80 percent threshold this project’s plan required before building a full loader. All twelve disagreements traced to one failure mode: the rule flagged a sentence as changed when a copy-edit or unrelated inserted sentence altered the surrounding text without touching the fact asked about. A reliable rule would need a proper wikitext renderer and paragraph-level alignment, estimated at eighteen to twenty-three additional hours the schedule did not allot. The decision to proceed with W1 alone, made once and not revisited, is disclosed as a limitation on external validity: every staleness result here comes from a self-generated workload, not an independently sourced ground truth.

W1 generates queries across several streams inside each topic cluster: canonical queries establishing a ground-truth answer, near-duplicate paraphrases sharing that answer’s identity, novel long-tail queries that never repeat, and time-shifted repeats of an existing answer. Every cluster carries its own randomly drawn half-life and regeneration-cost distribution. The generator guarantees that for every repeating answer identity, at least one later occurrence lands past that answer’s true expiry, since otherwise the validity gate would have nothing to reject. Each trace is split by arrival time into the first thirty percent for calibration and the remaining seventy percent for evaluation; no parameter is fit on data it is later scored against.

The generator’s two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and observed update-frequency in a public corpus. Neither was reachable from the development environment, so both distributions are instead parameterized from commonly cited public figures and compared against those figures as a plausibility check, documented as such rather than presented as a real external fit — a second explicit limit on external validity.

4.2 Simulation model

The benchmark harness every experiment runs through is a pure trace replayer; no language model is ever called. On a miss, response content is whatever answer identity the trace recorded, cost charged is the trace’s recorded regeneration cost, and miss latency is simulated from a seeded log-normal distribution scaled by entry size, deterministic per query so replay order never affects the result. This distribution is a documented placeholder, not fit to a real inference trace. Hit latency and the extension’s own overhead, cluster lookup plus gate check, are measured for real with wall-clock timing, not simulated. Reported latency is therefore a hybrid of real cache-side measurement and simulated backend cost, the right trade for making every run free and deterministic, but a validity limitation worth stating plainly.

A further limitation shapes every absolute number below: the harness’s index is an exact-text-match lookup, with no embedding-based semantic index yet wired in. Paraphrase queries, one of W1’s four stream types, never register as hits; only literal repeats do. Hit counts across

every experiment are in the low hundreds out of thousands of scored queries, widening every confidence interval. This caps absolute hit rate well below what a real semantic index would produce, but applies identically across every condition, so it does not favor one policy or lambda source over another; the relative comparisons this report’s claims rest on are unaffected, even though absolute numbers are not representative of a production cache.

The cache starts empty in every run. The first ten percent of the evaluation split warms the cache and is excluded from every metric; only the remaining ninety percent is scored.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry’s true expiry; false-hit-rate is the fraction where the served answer’s identity does not match the query’s true answer identity. Cost saved sums regeneration cost over non-stale hits only, since a stale hit served a wrong answer and saved nothing. Every configuration was run with ten independent seeds, each its own freshly generated trace rather than one trace replayed ten times, since the pipeline has no randomness once a trace is fixed. Results are medians with 95 percent confidence intervals from a percentile bootstrap over the ten per-seed values, ten thousand resamples, fixed resampling seed for exact reproducibility. Group comparisons use the Mann-Whitney U test; “significant” is used only where that test was run.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling

The first and most consequential experiment tests the core claim behind fitting a staleness rate per cluster at all: does it do work relative to the two obvious alternatives, a rate pooled across every cluster, and the true rate a real system could never observe. Three lambda sources, global, learned, and oracle, were each run for ten seeds on the same twelve-thousand-query W1 configuration, gate enabled, FreCoS eviction active throughout.

Table 1: Bracketing experiment: median stale-hit-rate by lambda source, 10 seeds each, with 95% bootstrap confidence intervals.

Lambda source	Median stale-hit-rate	95% CI	Median cost saved (USD)
Global	0.144	(0.085, 0.214)	0.448
Learned	0.035	(0.024, 0.060)	0.449
Oracle	0.035	(0.028, 0.064)	0.449

Learned separates cleanly from global. Table 1 shows a median stale-hit-rate of 0.035 under learned against 0.144 under global; the confidence intervals do not overlap, Mann-Whitney $p \approx 0.0005$. Pooling every cluster’s observations into one rate is measurably and significantly worse than fitting each cluster’s own.

Where this departs from what the design anticipated is the relationship between learned and oracle: the two are statistically indistinguishable, learned’s median equals oracle’s exactly, eight of ten seed pairs are byte-identical, $p \approx 0.94$. Learned does not sit at a visible midpoint between floor and ceiling; it sits at the ceiling. At roughly five hundred fifty to six hundred fifty calibration observations per cluster here, well above the thirty-observation fallback, the per-cluster maximum-likelihood fit essentially recovers the true rate, leaving no daylight for learned and oracle to differ on.

A follow-up tested whether that gap opens at a substantially smaller calibration sample, a trace roughly seven times smaller, dropping observations per cluster to the thirty-to-seventy range. It does not: learned remains indistinguishable from oracle ($p \approx 0.96$), while learned

versus global remains significant ($p \approx 0.0015$). The reason is a confound, not a property of the fit itself: shrinking the trace shrinks the calibration sample and the evaluation hit count together, and it is the eval-side hit count, not calibration sample alone, that determines whether estimation noise ever surfaces in a measured stale-hit-rate.

Figure 1 shows both runs on one axis. The finding, stated plainly rather than reframed to match the design’s anticipation: per-cluster fitting beats a single shared rate by a large, significant margin, and does so robustly across roughly a tenfold range of calibration sizes, tracking a rate the system could never observe. That is a stronger form of the underlying claim than “learned lands strictly between the two brackets” would have been, even though the literal bracket shape did not materialize.

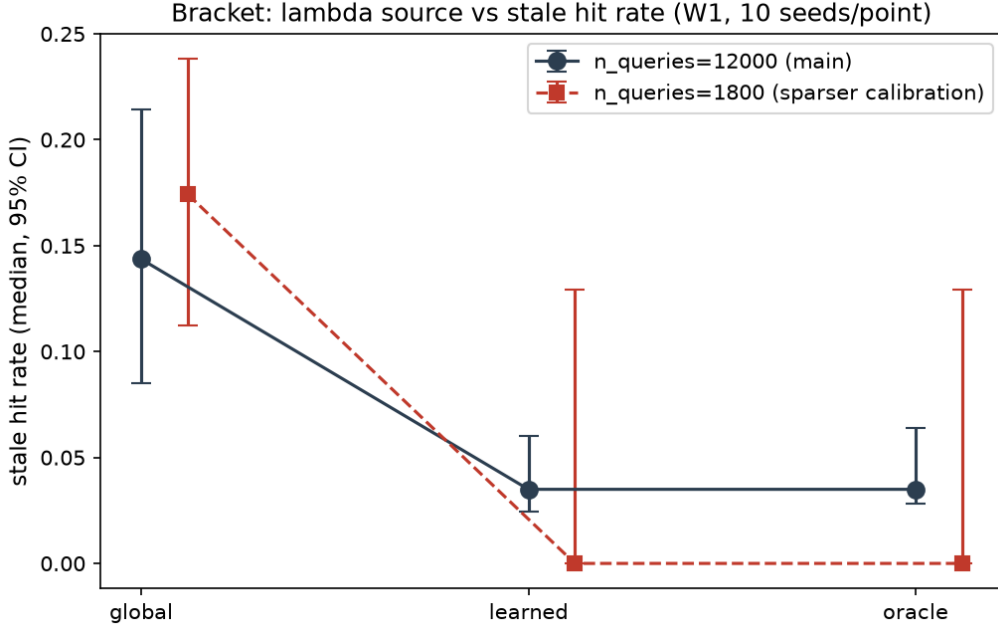


Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the original calibration scale and at a roughly sevenfold sparser calibration scale. Learned tracks oracle closely at both scales and separates significantly from global at both.

5.2 Ablation: isolating the gate and the eviction term

Six configurations were run for ten seeds each on the same workload: stock LRU with the gate disabled, stock LFU with the gate disabled (the floor of comparison per Biton and Friedman’s own finding that LRU underperforms on semantic workloads), a documented substitute for Biton and Friedman’s own released policy with the gate disabled, the validity gate combined with plain LFU eviction, the validity gate combined with full FreCoS eviction, and the validity gate combined with FreCoS eviction with the size-normalization term removed.

Against the LFU floor, median 0.680, the gate collapses stale-hit-rate roughly fifteenfold in every gated row (Table 2): gate-plus-LFU reaches 0.042, the full stack 0.035, the size-ablated variant 0.042. Hit rate drops by roughly a third, from 0.052 to about 0.017, the trade-off the design anticipated: the gate converts stale hits into misses by construction.

One row produced an unplanned but informative finding. The documented substitute for Biton and Friedman’s policy turned out identical to plain LFU in every correctness and cost column, seed by seed, to full precision. This traces to a real workload property, not a bug: at this configuration’s roughly five percent hit rate, almost every entry at eviction time has never been accessed a second time, so the substitute’s frequency-divided-by-cost score evaluates to zero for essentially every candidate, and the policy falls through to LFU’s own tie-break. The

Table 2: Ablation: median stale-hit-rate and hit rate by row, 10 seeds each. Row 2 (LFU, gate off) is the floor of comparison.

Row	Configuration	Stale-hit-rate	Hit rate	Δ stale-hit-rate vs. row 2
1	LRU, gate off	0.661	0.048	−0.019
2	LFU, gate off (floor)	0.680	0.052	—
3	B&F substitute, gate off	0.680	0.052	+0.000
4	LFU, gate on	0.042	0.017	−0.638
5	FreCoS, gate on (full stack)	0.035	0.017	−0.645
6	FreCoS (no size), gate on	0.042	0.017	−0.638

primary eviction comparator is therefore effectively untested by this workload’s parameters, a limitation stated here rather than concealed.

The predicted interaction, that the gate carries most of the improvement with the decay term contributing less on top than alone, held more strongly than anticipated. Isolating the gate’s own effect (gate-plus-LFU against the floor) gives -0.638 ; isolating FreCoS’s decay term on top of an active gate gives a further -0.007 , an order of magnitude smaller and not statistically distinguishable from zero ($p \approx 0.97$, against $p \approx 0.0002$ for the gate’s own effect). The gate accounts for roughly ninety-nine percent of the total move from floor to full stack. This is not the contradiction flagged as worth investigating in advance, since the predicted direction held; both components were checked for a shared cluster-identity bug regardless, and neither disagreed with the other about an entry’s cluster.

The size-normalization row was likewise indistinguishable from the full stack and from gate-plus-LFU on every metric. With the gate already filtering out anything stale, the entries eviction chooses among are uniformly fresh, leaving the decay term little to differentiate on; the size term’s effect is plausibly more visible under tighter cache-size pressure, which the cache-size sweep below investigates.

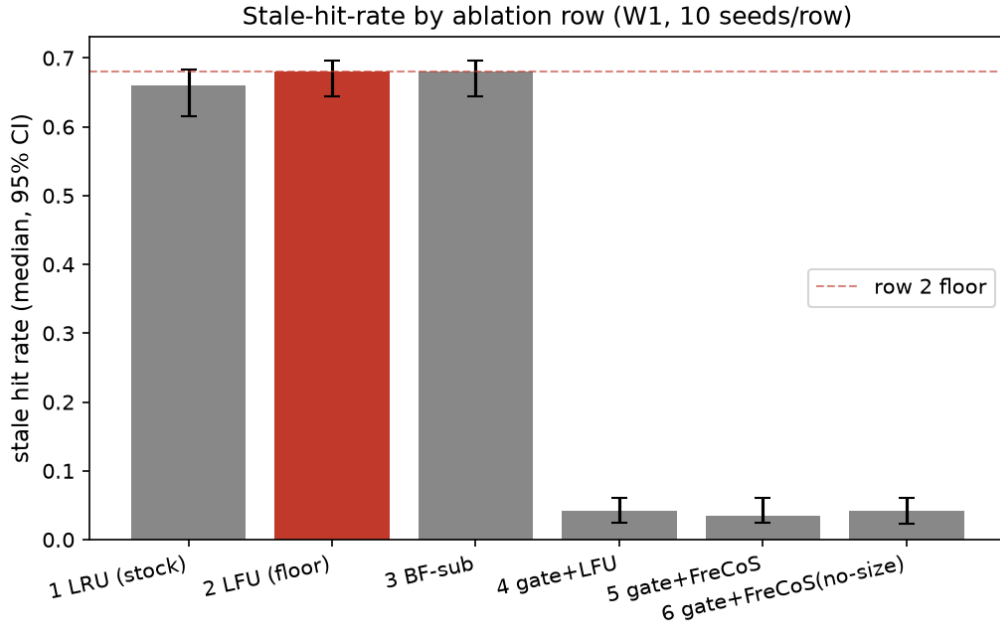


Figure 2: Stale-hit-rate across all six ablation rows, with 95 percent confidence intervals. Row two, LFU with the gate disabled, is marked as the floor of comparison.

5.3 Cache-size sweep and the correctness-efficiency trade-off

Cache size was swept across five points spanning five to eighty percent of the workload’s distinct-answer working set, full stack throughout. Hit rate rises from 0.013 at the smallest point to 0.017 at thirty percent, then flattens completely; the three largest points produce identical per-seed hit counts. The knee, identified where marginal hit-rate gain per additional entry drops by more than an order of magnitude relative to the previous step, falls at thirty percent: the slope drops roughly thirteenfold into that point and is exactly zero afterward. This cache size anchors every other sweep in this report.

The trade-off curve this project’s design calls its honest core comes from sweeping the gate’s target confidence level while holding cache size fixed at that knee.

Table 3: TTL confidence sweep: median stale-hit-rate and hit rate as target confidence varies, 10 seeds each.

TTL confidence	Median stale-hit-rate	Median hit rate
0.80	0.126	0.019
0.90	0.042	0.017
0.95	0.023	0.017
0.99	0.008	0.016

As confidence rises from 0.80 to 0.99 (Table 3), stale-hit-rate falls from 0.126 to 0.008, a sixteenfold reduction, while hit rate falls from 0.019 to 0.016, fifteen percent. The exchange is not uniform: most of the stale-hit-rate benefit is captured in the first step, 0.80 to 0.90, where it falls from 0.126 to 0.042, while hit rate erodes at a steadier pace across the full range. The cheapest correctness gains come early; pushing toward 0.99 buys a shrinking reduction in staleness for roughly the same ongoing hit-rate cost. This is a trade-off curve, not evidence a stricter gate is uniformly better: an operator chooses a point on it to suit their own tolerance for stale answers against misses.

A fourth sweep varied cluster count at five, ten, twenty, and fifty. This came out non-monotone on both metrics, with every adjacent confidence interval overlapping heavily, so no directional effect is distinguishable from noise at ten seeds per point. A plausible mechanism: more clusters shrinks each one’s calibration sample, making the learned rate noisier, while finer-grained clusters could in principle track heterogeneous half-lives more precisely, two effects pulling in opposite directions. This result sits in a supplementary table rather than a headline figure, since it clears neither the effect-size nor confidence-interval bar the rest of this report’s figures were held to.

6 Discussion

The headline claim this project set out to test, that fitting a staleness rate from observed data does measurable work beyond ignoring cluster structure or assuming a single global rate, is supported clearly and repeatedly: the learned rate beats naive pooling with a large, significant margin in every experiment that varied the lambda source, whether calibration data is abundant or an order of magnitude sparser. Learned tracking oracle rather than sitting at a visible midpoint is a stronger result for the underlying claim than the originally anticipated shape, even though it does not match the literal bracket picture predicted in advance; reporting it as the actual finding rather than reframing it is deliberate, since the alternative would misrepresent the data.

The second finding, that the gate rather than the decay term carries almost the entire improvement, is consistent with the design’s intended division of labor: the gate is the correctness

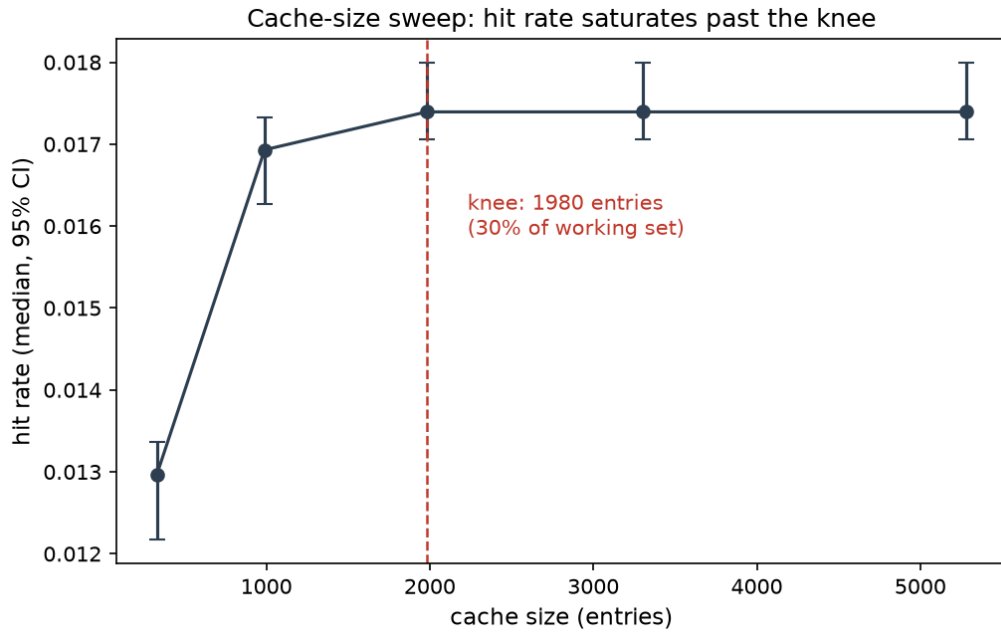


Figure 3: Hit rate against cache size, with the identified knee at thirty percent of the working set marked.

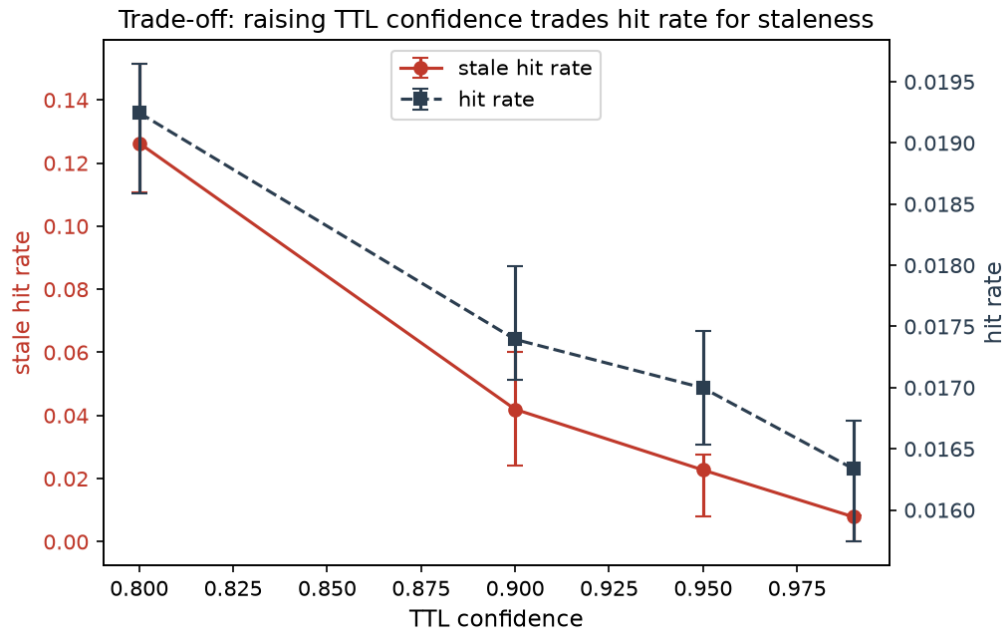


Figure 4: Stale-hit-rate and hit rate plotted together against the gate's target confidence level, showing the shape of the correctness-efficiency exchange rate rather than a single winning configuration.

mechanism, the eviction term only a soft prior among entries already judged fresh. The ablation confirms that division more sharply than the design anticipated. A system wanting only the staleness-correctness gain without the added eviction complexity could plausibly adopt the gate alone.

The correctness-efficiency trade-off is the part of this report that most needs to be read as a trade-off, not a win. Raising the gate’s target confidence buys a large reduction in stale-hit-rate at a comparatively small, real cost in hit rate, and that exchange is front-loaded: most of the gain is available cheaply, further gains cost proportionally more in foregone hits. A system adopting this gate chooses a point on the curve according to how much a stale answer costs relative to a miss; this report does not claim a universally correct point.

Two limitations bound how far these results generalize. First, every result comes from a synthetic, self-authored workload; the attempt to source external staleness ground truth from Wikipedia was made in good faith and abandoned on measured evidence, not convenience, but the limitation stands regardless. Second, the harness’s exact-match index caps every absolute rate in this report well below a real semantic index, and every finding above is a relative comparison that shares this limitation identically across conditions. A reader should treat every rate here as directionally informative and every absolute value as an artifact of this harness, not a production estimate.

7 Conclusion and future work

This project set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and to build and measure a system that closes it without abandoning cost-awareness in eviction. The gate reliably converts most stale hits into misses at a bounded, quantified cost in hit rate; the learned per-cluster rate that feeds it outperforms naive pooling significantly and consistently, tracking an unreachable oracle rate across an order of magnitude of calibration sizes; and the eviction value function’s cost and size terms behave correctly under every property test, even though their marginal contribution on top of an active gate is smaller than the design predicted.

Several directions were deliberately set aside as grounded future work. A real semantic index in place of the exact-match harness would likely change every absolute number reported and is the change most likely to make results comparable to production; the relative findings are expected to be directionally robust to it, but this was not verified. Calibrating the generator’s distributions against a real inference trace and content-update corpus, rather than the literature-informed proxies used here, would strengthen external validity. Dynamic re-clustering under drift, a probabilistic staleness budget in the style of FreshCache, and admission control layered on the current policy were all named as future work in the original design and remain so.

Appendix: code and data artifacts

All code, tests, and results referenced in this report are in the project’s git repository. The extension package lives under `gptcache_ext/`, split into the serve-path pipeline seam (`pipeline.py`), additive metadata storage (`metadata.py`), the staleness model (`staleness/clusters.py`, `staleness/fitter.py`, `staleness/gate.py`), and eviction (`eviction/frecos.py`, `eviction/baselines.py`). The synthetic workload generator is under `workloads/w1_synthetic/`. The benchmark harness and its metrics are under `benchmarks/harness.py` and `benchmarks/metrics.py`; the four experiment runners referenced in Section 5 are under `benchmarks/runners/`. Every experiment’s raw results and its own written summary are under `results/`, organized by experiment (`results/brackets/`, `results/ablation/`, `results/sweeps/`). Every figure in this report is regenerated from those results by a single command,

```
cd analysis && python3 make_figures.py
```

with no hand-edited figure anywhere in this project. The reference oracle used to validate the pipeline’s decision logic and the callable invariant suite run on every benchmark execution are under `tests/oracle/` and `tests/invariants.py`. The GPTCache commit this project vendors and every source-code claim made about its behavior in Section 2 are recorded in `docs/baseline-source-map.md`. The Wikipedia feasibility spike discussed in Section 4.1 is recorded in full in `docs/w2-feasibility.md`.

References

- [1] N. Biton and R. Friedman. *From Exact Hits to Close Enough: Eviction Policies for Semantic Caches*. arXiv:2603.03301, 2026.
- [2] Cortex authors. *Cortex: A Cost-Aware Semantic Caching System for LLM Serving*. arXiv:2509.17360, 2026. NSDI 2026; earlier preprint circulated as “Asteria”.
- [3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.
- [4] Category-Aware authors. *Category-Aware Semantic Caching for Large Language Model Serving*. arXiv:2510.26835, 2025.
- [5] FreshCache authors. *FreshCache: Probabilistic Staleness Budgets for Retrieval-Augmented Generation*. arXiv:2607.04281, 2026.
- [6] SCALM authors. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.
- [7] MeanCache authors. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.
- [8] G. Einziger and R. Friedman. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.
- [9] vCache authors. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.
- [10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. <https://github.com/zilliztech/GPTCache>, 2024. Version 0.1.44, commit bae7ffef774e762d9d4e60fce70be00011188a6.
- [11] P. Rajpurkar, R. Jia, and P. Liang. *Know What You Don’t Know: Unanswerable Questions for SQuAD*. arXiv:1806.03822, 2018.